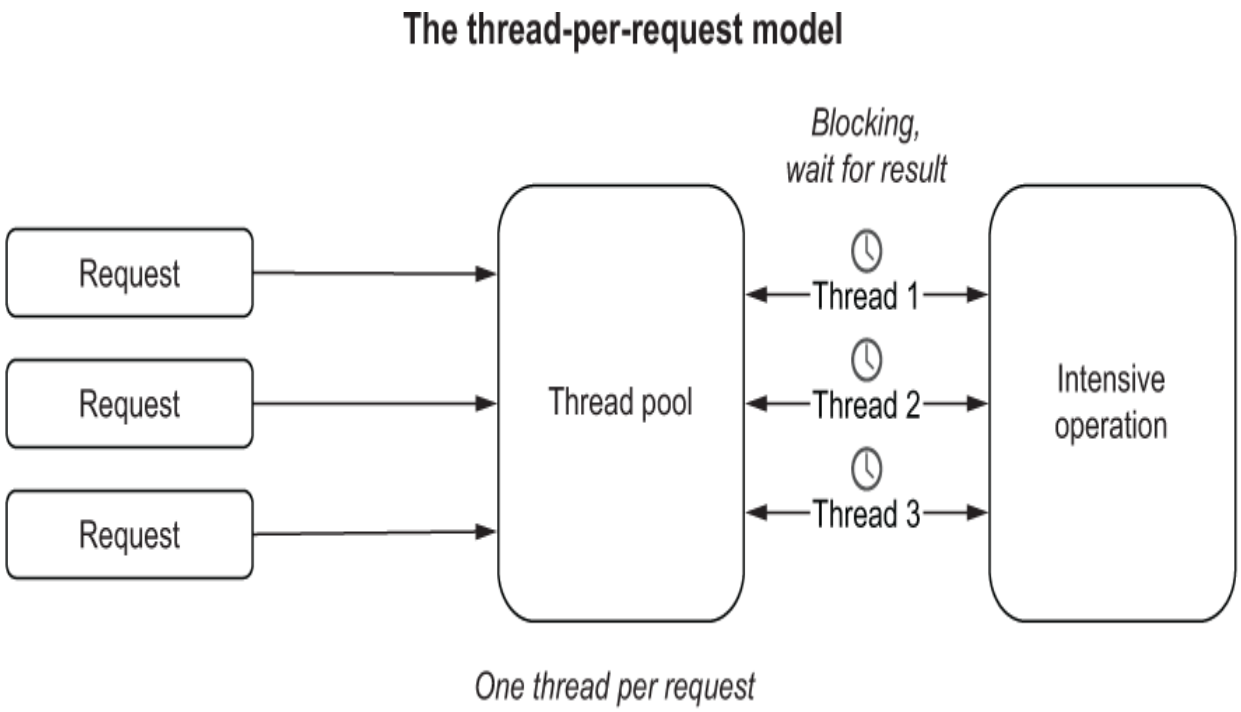


Figure 8.1 In the thread-per-request model, each request is handled by a thread dedicated exclusively to its handling.

Reactive applications are more scalable and efficient by design. Handling requests in a reactive application doesn't involve allocating a given thread exclusively—requests are fulfilled asynchronously based on events. For example, if a database read is required, the thread handling that part of the flow will not wait until data is returned from the database. Instead, a callback is registered, and whenever the information is ready, a notification is sent, and one of the available threads will execute the callback. During that time, the thread that requested the data can be used to process other requests rather than waiting idle.

This paradigm, called *event loop*, doesn't set hard constraints on the application's scalability. It actually makes it easier to scale, since an increase in the number of concurrent requests does not strictly depend on the number of threads. As a matter of fact, a default configuration for reactive